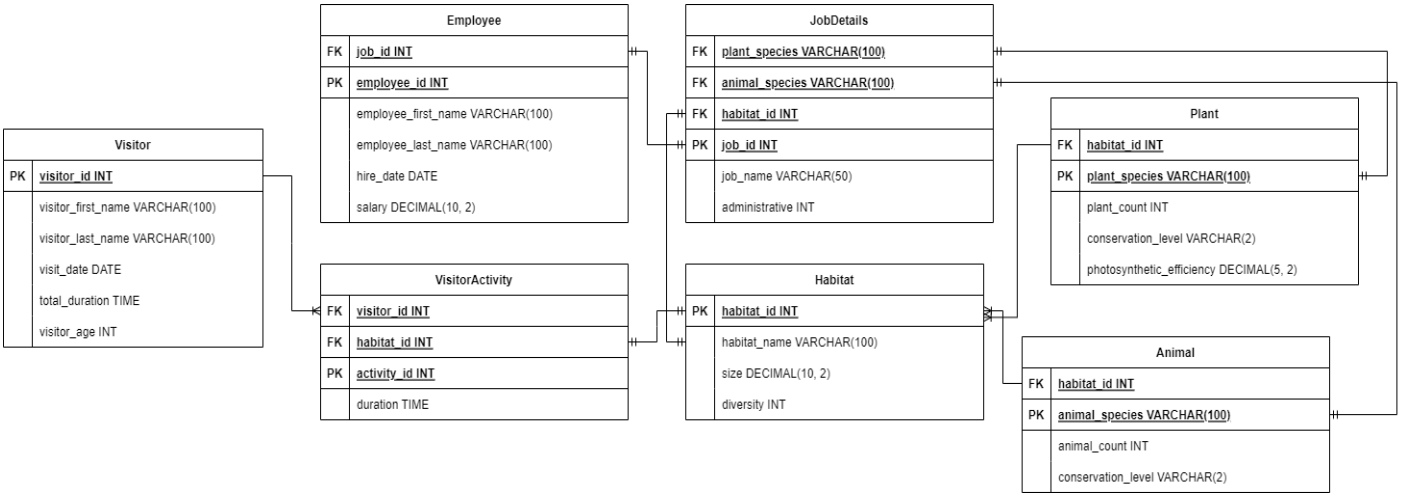
Descrierea Temei

1. Descrierea Bazei de Date

Proiectul ales este o baza de date pentru un parc ecologic. Aceasta tine evidenta habitatelor, faunei, florei, angajatilor, vizitatorilor, si activitatilor acestora.

1.1 Diagrama Bazei de Date



1.2 Structura Tabelelor

Tabelele de plante si animale contin informatii despre specii cum ar fi numarul de indivizi, habitatul in care traiesc (in relatie one to one cu tabelul Habitat), si date specifice precum nivelul de conservare al speciei sau eficienta fotosintetica. Tabelul cu Habitate contine id-ul unic, numele, suprafata acestuia si diversitatea – adica numarul de specii diferite din acesta. Tabelul JobDetails contine informatii despre posturi: de ce habitate, animale, sau plante e responsabil (daca exista) si daca postul este administrativ sau nu. Tabelul Employee contine date despre fiecare angajat, inclusiv postul acestuia (in relatie one to one cu JobDetails). Tabelul Visitor contine date despre toti visitatorii precum data in care au vizitat, cat au stat in parc, precum si date personale despre acestia. In tabelul Visitor Activty se afla date despre ficare activitate a unui vizitator precum habitatul in care s-a petrecut (in relatie one to one cu tabelul Habitat) si durata activitatii.

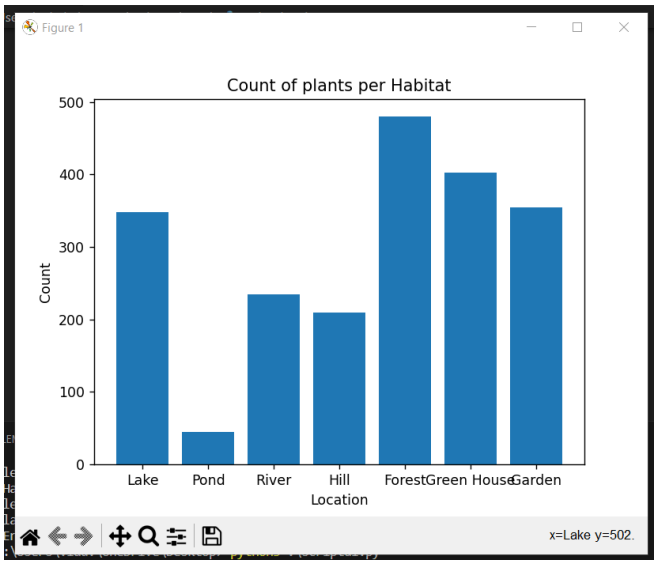
1.3 Descrierea Constrangerilor de Integritate

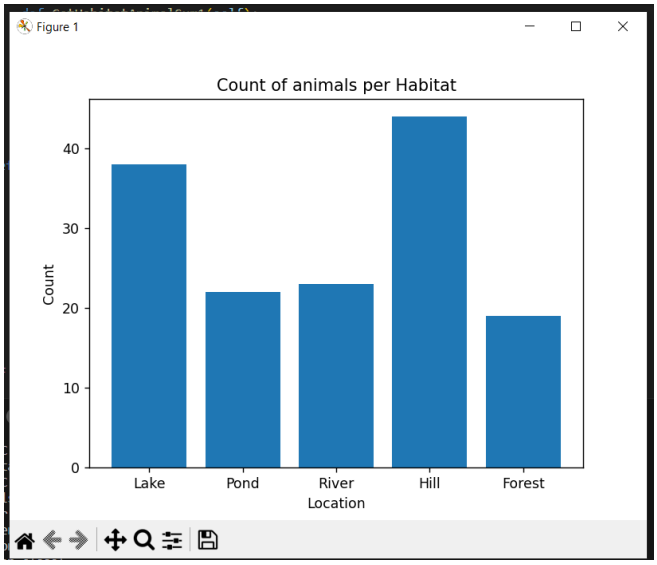
O serie de atribute precum Habitat size, Employee salary si Visitor Age trebuie sa fie mai mari strict decat 0. Atribute precum Habitat Diversity, Animal si Plant count trebuie sa fie mai mari sau egale ca zero. Urmand un format standard, nivelul de conservare al fiecarei specii poate luat o valoare dintr-o lista de 7 acronime. Fiind de tip boolean, valoarea administrative din cadrul unui post poate lua doar valorile 1 sau 0. Fiind un procent, valoarea photosynthetic efficiency va avea o valoare cuprinsa intre 0 si 1.

Am creat urmatoarele triggere:
Pentru inserarea unei noi specii de animale sau plante, diversitatea habitatului in care acestea se afla este incrementat. In cazul stergerii sau schimbarii nivelului de conservare la ‘EX’ (extinct) al unei specii, valoarea este decrememntata. In cazul mutarii inei specii intr-un alt habitat (UPDATE), valoarea diversitatii noului habitat creste iar cea a vechiului habitat scade. In cazul adaugarii unei noi activitati, timpul total al respectivului utilizator creste cu timpul petrecut in acea activitate.

1.4 Descrierea Procedurilor si Functiilor

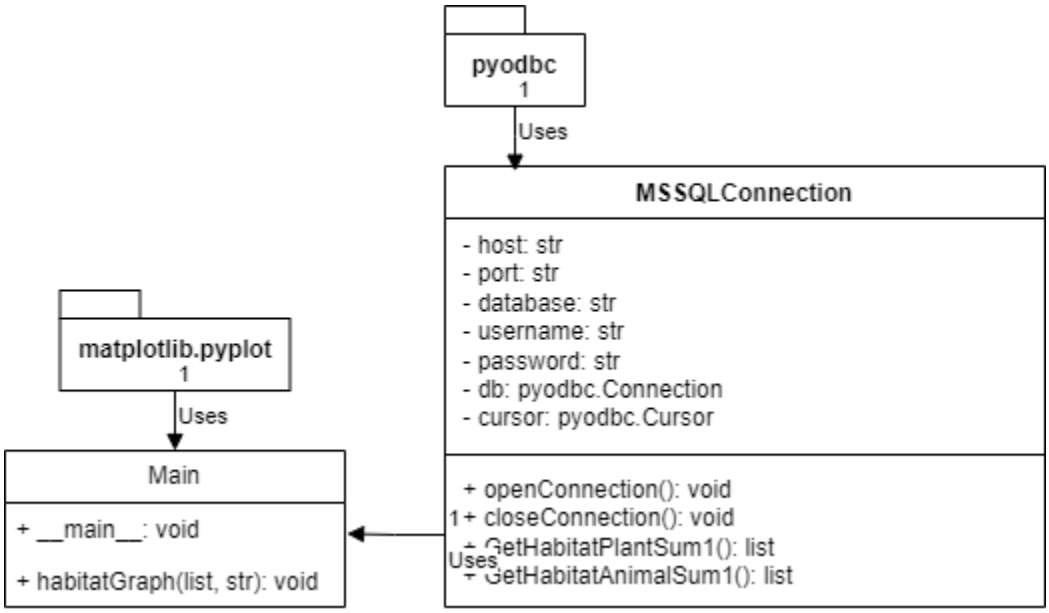
Am implementat doua proceduri de complexitate 4. Prima creaza un tabel temporar cu numele habitatelor si numarul de specii de animale. In continuare, se face suma tuturor animal_count din acelasi habitat si se adauga in tabelul temporar. Se unesc tabelele Habitat si Animal pe baza id-ului habitatului, asigurand-se ca numarul de animale nu este null. Rezultatul este grupat dupa id-ul si numele habitatului. In cazul in care se doreste filtrarea habitatelor dupa un numar minim de indivizi, acest lucru se poate realiza cu variabila @minTotalAnimalCount. Am folosit SET NOCOUNT ON pentru a imbunatati performanta procedurii. Cea de a 2-a procedura returneaza un tabel cu numele habitatelor si numarul de specii de plante din habitatul respectiv. Implementarea este similara cu procedura interioara.





2. Descrierea Aplicatiei

2.1 Diagrama de Clase

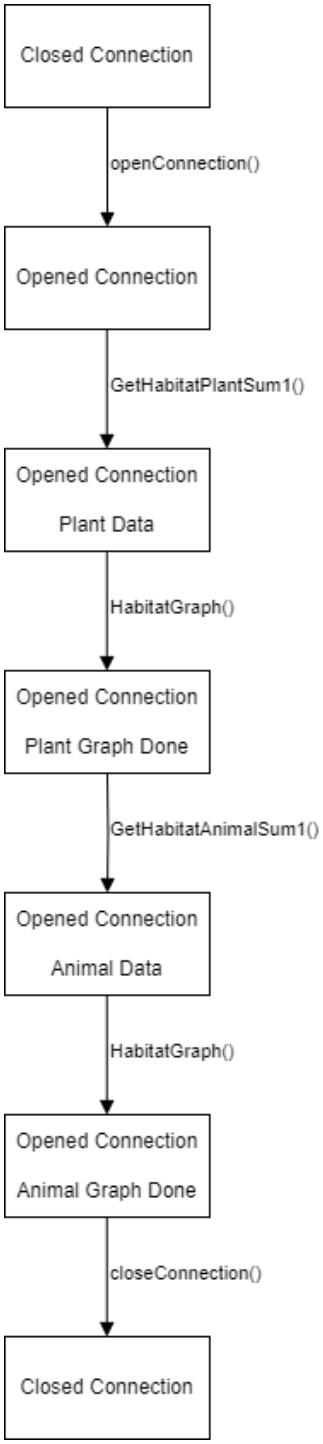


2.2 Structura Claselor

Clasa principala este MSSQLConnection. Atributele host, port, database, username, password si db au ca scop stabilirea conexiunii cu baza de date. Atributul cursor este folosit pentru apelarea procedurilor. Metodele openConnection si closeConnection au ca scop conectarea, respectiv deconectarea de la baza de date iar metodele GetHabitatPlantSum1 si GetHabitatAnimalSum1 returneaza tabelele generate de cele doua proceduri. Aceasta clasa se foloseste de biblioteca pyodbc.

In main se face apelarea metodelor din clasa MSSQLConnection iar in functia habitatGraph se genereaza grafice pe baza outputurilor metodelor GetHabitatAnimalSum1 si GetHabitatPlantSum1. Cea din urma se foloseste de biblioteca matplotlib.pyplot pentru crearea de barcharts.

2.3 Diagrama de stari si workflowul aplicatiei



2.4 Conexiunea aplicatiei la baza de date

Conexiunea la baza de date se face prin intermediul metodei openConnection din clasa MSSQLConnection. Astfel, cu ajutorul functiei pyodbc.connect ce primeste ca parametru in sir de caracter care, pe langa parametri din diagrama, foloseste si parametri Driver, Encrypt, TrustServerCertificate si Trusted_Connection, se realizeaza conexiunea la serverul local cu baza de date si la baza de date. Cu ajutorul unui cursor se executa procedurile prin intermediul cuvantului cheie EXEC, numelui procedurii si parametrii sai necesari. Conexiunea este inchisa cu ajutorul metodei closeConnection.

3. Capturi de eclan pentru interfete si rapoarte

```sql
USE parc_eco_db;
GO

CREATE PROCEDURE GetHabitatAnimalSum1
    @minTotalAnimalCount INT = 0
AS
BEGIN
    SET NOCOUNT ON;
    CREATE TABLE #HabitatAnimalSum (
        HabitatName NVARCHAR(100),
        TotalAnimalCount INT
    );
    INSERT INTO #HabitatAnimalSum (HabitatName, TotalAnimalCount)
    SELECT
        h.habitat_name,
        ISNULL(SUM(a.animal_count), 0) AS total_animal_count
    FROM
        Habitat h
    LEFT JOIN
        Animal a ON h.habitat_id = a.habitat_id
    WHERE
        a.animal_count IS NOT NULL
    GROUP BY
        h.habitat_id, h.habitat_name
    HAVING
        ISNULL(SUM(a.animal_count), 0) >= @minTotalAnimalCount;

    SELECT * FROM #HabitatAnimalSum;

    DROP TABLE #HabitatAnimalSum;
END;
```

```sql
USE parc_eco_db;
GO

CREATE PROCEDURE GetHabitatPlantSum
    @minTotalPlantCount INT = 0
AS
BEGIN
    SET NOCOUNT ON;
    CREATE TABLE #HabitatPlantSum (
        HabitatName NVARCHAR(100),
        TotalPlantCount INT
    );

    INSERT INTO #HabitatPlantSum (HabitatName, TotalPlantCount)
    SELECT
        h.habitat_name,
        ISNULL(SUM(p.plant_count), 0) AS total_plant_count
    FROM
        Habitat h
    LEFT JOIN
        Plant p ON h.habitat_id = p.habitat_id
    WHERE
        p.plant_count IS NOT NULL
    GROUP BY
        h.habitat_id, h.habitat_name
    HAVING
        ISNULL(SUM(p.plant_count), 0) >= @minTotalPlantCount;

    SELECT * FROM #HabitatPlantSum;

    DROP TABLE #HabitatPlantSum;
END;
```

4. Concluzii

Proiectul a fost interesant si am apreciat varietatea taskurilor raportata la volumul de munca. De asemenea, am reusit sa intelege intr-un mod mai profund atat workflowul in crearea unei baze de date, cat si crearea unei aplicatii ce utilizeaza o baza de date.

5. Bibliografie

Laboratoare:
https://ocw.cs.pub.ro/courses/bd2/laboratoare/07
https://ocw.cs.pub.ro/courses/bd2/laboratoare/08
https://ocw.cs.pub.ro/courses/bd2/laboratoare/09
Microsoft Server:
https://learn.microsoft.com/en-us/sql/connect/python/pyodbc/step-1-configure-development-environment-for-pyodbc-python-development?view=sql-server-ver16&tabs=windows

https://learn.microsoft.com/en-us/sql/connect/odbc/download-odbc-driver-for-sql-server?view=sql-server-ver16#download-for-windows